LATENT WORLD MODELS SHOULD PLAN WITH DISTANCES ALONG THE LATENT MANIFOLD

Zunzhi You[1], **Linwei Tao**[1,2], **Siyu Xu**[1], **Yunke Wang**[1], **Chang Xu**[1]
[1]The University of Sydney [2]Machine Learning Research, Apple
zyou5834@uni.sydney.edu.au
{linwei.tao,s.xu,yunke.wang,c.xu}@sydney.edu.au

ABSTRACT

Latent world models such as LeWM plan by searching for an action sequence whose predicted terminal latent is closest to the goal latent under an ambient norm, such as the Euclidean distance. Planning with such a cost assumes that proximity in the ambient latent space reflects proximity to the goal in the environment. Studying LeWM, we find that this assumption holds only in a small neighborhood around the goal, whereas farther away the L2 cost forms spurious low-cost basins that trap the planner. Yet neighborhood-based embeddings show that the latents retain the underlying physical state. We therefore hypothesize that the latents lie near a curved, low-dimensional manifold, and that the terminal cost should measure the distance along this latent manifold rather than through the ambient space. To estimate this manifold distance, we propose the Neighborhood Graph Cost (NGC), which computes shortest-path distances on a mutual nearest-neighbor graph constructed over training latents. NGC changes only the terminal cost and requires no retraining. This single change improves the planning success rate of LeWM across all four benchmark tasks and nearly solves TwoRoom and Reacher: even at the farthest goal offsets tested, NGC reaches 95% and 100% success, whereas the L2 cost falls to 27% and 81%. These results support the manifold hypothesis and our argument that latent world models should plan with distances along the latent manifold.

1 INTRODUCTION

In the paradigm of planning with world models, an agent predicts the outcomes of candidate action sequences and executes the sequence whose predicted outcome scores best (Finn & Levine, 2017; Hafner et al., 2019; LeCun, 2022; Hansen et al., 2024). When the task is specified by a goal observation, the score is naturally a distance between the predicted outcome and the goal, without requiring a separately designed reward. Because pixel-wise distances to a goal image are dominated by large objects such as the robot arm (Ebert et al., 2018), latent world models measure the distance in a learned representation space instead (Zhou et al., 2025; Sobal et al., 2025; Assran et al., 2025; Terver et al., 2026; Maes et al., 2026b). These models typically score the predicted terminal latent against the goal latent using a norm of their difference, such as the Euclidean distance (Maes et al., 2026b) or the L1 distance (Assran et al., 2025), serving as the sole signal guiding the planner toward the goal. Planning with such a cost therefore assumes that a latent closer to the goal latent in the latent space corresponds to a state closer to the goal in the environment.

In this work, we study LeWM (Maes et al., 2026b), an end-to-end latent world model framework that uses the squared Euclidean distance as its terminal cost (the L2 cost), and find that this assumption fails even on a simple task. In TwoRoom, a point agent moves from one room to the other through a doorway, and the observation is determined solely by the agent's 2D position. Yet LeWM reaches a success rate of only about 90% when the goal observation is 25 steps from the start, dropping below 30% at 75 steps. In every failed episode, the agent becomes trapped in a region far from the goal. The cost maps in Figure 1 (top) reveal why: while the L2 cost forms a true basin around the goal, it also creates spurious low-cost basins far away that trap the planner.

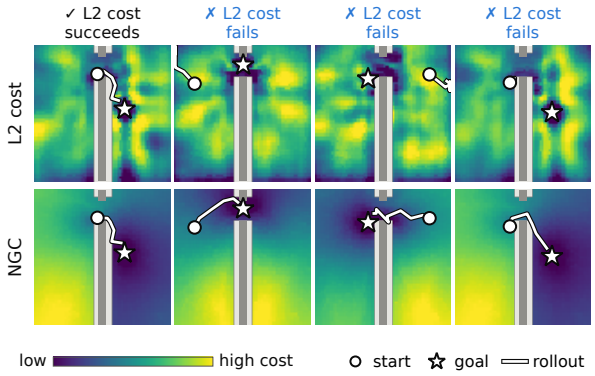


Figure 1: Cost maps and rollout trajectories for four start–goal pairs in TwoRoom. **Top:** the L2 cost (Equation 1) between the latent rendered at each feasible grid position and the goal latent. **Bottom:** NGC (Section 4) on the same pairs. Details in Appendix B.1.

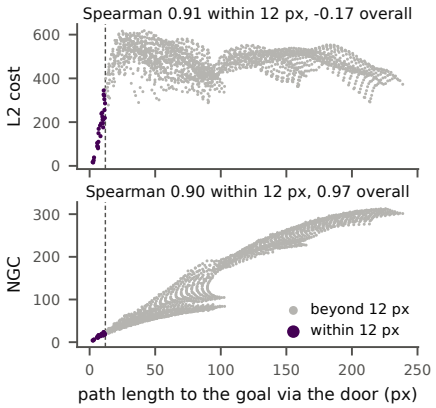


Figure 2: Terminal cost against the length of the shortest feasible path to the goal, for every feasible grid position of the rightmost pair in Figure 1. **Top:** the L2 cost. **Bottom:** NGC.

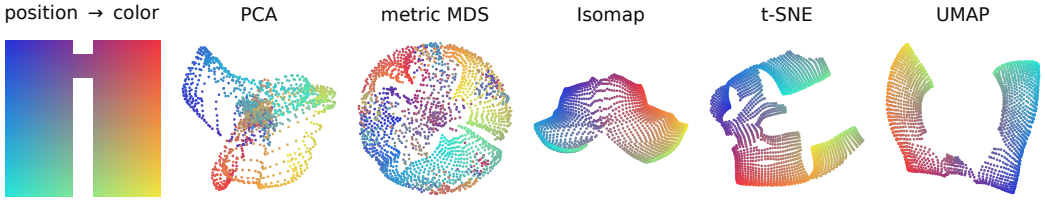


Figure 3: Two-dimensional embeddings of the TwoRoom grid latents, with each point colored by its rendered physical position (**left:** color key).

To understand these basins, we compare the L2 cost against the physical distance to the goal and find that it tracks this distance only locally, ceasing to increase with it farther away (Figure 2). Yet neighborhood-based embeddings such as Isomap (Tenenbaum et al., 2000) unfold the latents into a faithful two-dimensional map of the agent's position, whereas Euclidean-distance-based embeddings fold the space onto itself (Figure 3). The planning failures, therefore, stem not from the latents losing task-relevant state information, but from how they are read.

These observations lead us to hypothesize that the latents of a trained LeWM lie near a curved, low-dimensional manifold, and that the terminal cost should measure the distance along this latent manifold rather than through the ambient latent space. To estimate this manifold distance, we propose the **Neighborhood Graph Cost** (NGC). NGC connects the latents of training observations to their mutual nearest neighbors, attaches the goal latent and each predicted terminal latent to the graph, and scores a candidate by the shortest-path length between the two. On the TwoRoom pairs, NGC produces a single clean basin at the goal (Figure 1, bottom).

We compare NGC with the L2 cost across all four LeWM tasks. NGC achieves a success rate of 94.6% or higher on TwoRoom and 99.4% on Reacher, whereas the L2 cost drops to 27% and 81% at the farthest goal offsets. On Cube and PushT, NGC improves the success rate at the farthest offsets by 16 and 15 percentage points. This gain comes from the cost formulation itself: NGC still outperforms the L2 cost when paired with an oracle predictor that executes every candidate in the simulator, confirming that it does not merely compensate for errors of the learned predictor. Moreover, the L2 cost fails to match NGC even with four times as many CEM candidates.

In summary, we offer a simple explanation for the planning failures we observe in LeWM. It leads to a method that changes only the terminal cost, and the method's effectiveness in turn supports

the explanation. We hope that our work leads the latent world model community to reconsider the default of measuring distances through the ambient latent space, and to plan with distances along the latent manifold instead.

2 BACKGROUND AND RELATED WORK

Latent world models for goal-conditioned planning. A latent world model encodes observations into a learned latent space and predicts forward in that space, allowing a planner to search over action sequences without rendering pixels (Hafner et al., 2019; LeCun, 2022). When the task is specified by a goal observation, the planner scores each candidate action sequence by a terminal cost between the predicted terminal latent and the goal latent. DINO-WM (Zhou et al., 2025) plans in frozen DINOv2 features; PLDM (Sobal et al., 2025) plans in a latent space trained jointly with the predictor; V-JEPA 2 (Assran et al., 2025) and Terver et al. (2026) plan with JEPA-family world models. In all cases, the terminal cost is a norm of the latent difference, such as the Euclidean or L1 distance.

We study LeWM (Maes et al., 2026b;a), which trains an encoder E and a predictor F end-to-end. The encoder maps an observation o_t to a latent $z_t = E(o_t)$, and the predictor maps a short history of latents and actions to the next latent, $\hat{z}_{t+1} = F(z_{t-2:t}, a_{t-2:t})$. By rolling F forward H steps from the current latent under a candidate action sequence $(a_t, \ldots, a_{t+H-1})$, the planner obtains a predicted terminal latent $\hat{z}_{t+H}$. The planner uses CEM (Rubinstein, 1999; de Boer et al., 2005): it samples candidate action sequences, ranks them by the terminal cost, and iteratively refines the sampling distribution toward lower-cost candidates. Writing $q = \hat{z}_{t+H}$ for the predicted terminal latent and $g = E(o_{\text{goal}})$ for the goal latent, the terminal cost is

$$C_{\text{L2}}(q, g) = \|q - g\|_2^2. \tag{1}$$

Because CEM ranks candidates solely by this cost, planning with the L2 cost assumes that a shorter Euclidean distance to the goal latent reflects progress toward the goal in the environment.

Improving LeWM planning. Many follow-ups to LeWM aim to improve its planning, largely motivated by the observation that the Euclidean distance between latents fails to measure progress toward the goal. Their remedies fall into three families. The first *retrains the encoder* so that the distance between latents itself measures progress, whether by straightening trajectories (Wang et al., 2026b), matching latent to ground-truth state distances (Hu et al., 2026), adding reachability or trajectory heads (Li et al., 2026b; Wang et al., 2026a; Bai & Xiong, 2026; Huang et al., 2026), shaping the latent space with a value function (Destrade et al., 2026), or embedding in a hyperbolic space (Liu et al., 2026). The second *learns a parametric cost* on the frozen model, such as a temporal-separation regressor (Li et al., 2026a; Singh, 2026), a goal-conditioned value (Sommer & Schilling, 2026), or a ground-truth state readout (Pham & Bera, 2026). The third *changes the target* rather than the cost, planning toward a learned subgoal (Masip et al., 2026; Caselli et al., 2026; Cheng et al., 2026) or replacing the search with a learned actor (Sun et al., 2026). Each adds a learned component that must be trained beyond the original LeWM objective. We depart from all three by attributing the issue to the latents lying near a curved manifold and proposing a method to estimate the distance along it, without training a new encoder, a cost network, a subgoal generator, or a policy.

Manifold learning and shortest-path distances. NGC estimates distances along the manifold using ideas from manifold learning. Isomap (Tenenbaum et al., 2000) approximates the geodesic distance along a data manifold by the shortest path on a k-nearest-neighbor graph weighted by Euclidean edge lengths, then embeds the resulting distance matrix with multidimensional scaling. t-SNE (van der Maaten & Hinton, 2008) and UMAP (McInnes et al., 2018) optimize different embedding objectives on similar graphs. NGC builds the same kind of graph over encoder outputs, but it never computes embeddings or coordinates. A separate line of work builds graphs over experience and plans by searching them for waypoints (Savinov et al., 2018; Eysenbach et al., 2019; Yang et al., 2020; Zhang et al., 2021). There, the edges encode demonstrated transitions or learned reachability, and the output is a plan. In contrast, NGC's edges encode only Euclidean proximity, and they are used to score candidates rather than to generate action sequences.

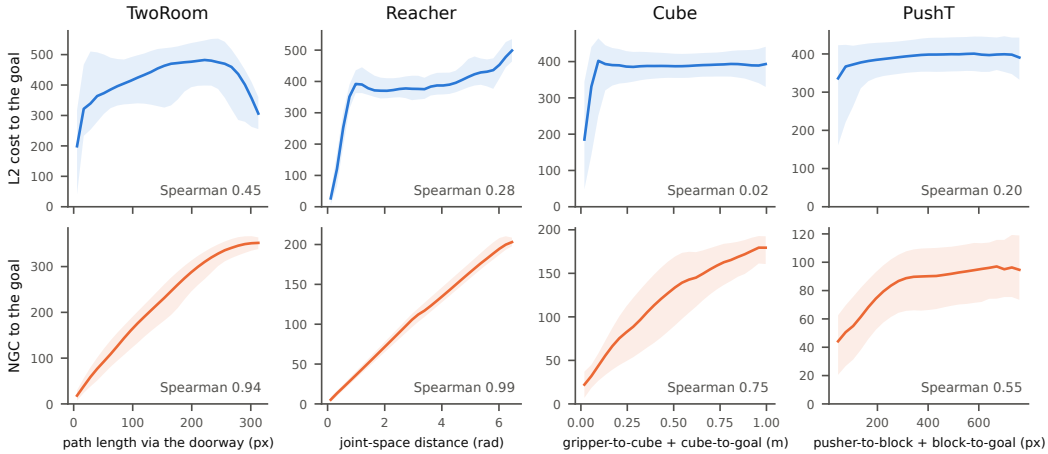


Figure 4: Terminal cost against the physical distance to the goal across all four tasks. **Top:** the L2 cost. **Bottom:** NGC. Lines show bin medians, and shaded bands indicate the 10th to 90th percentiles. Details in Appendix B.2.

3 WHY LEWM DOES NOT SOLVE TWOROOM

We begin by examining how LeWM performs on TwoRoom. Of the four tasks in LeWM, TwoRoom appears to be the easiest: a point agent moves from one room to the other through a fixed doorway, so the top-down observation is determined by the agent's 2D position alone. Yet under our evaluation protocol (Section 5.1), LeWM reaches a success rate of only about 90% on TwoRoom, compared with 100% on Reacher and 91% on PushT. Because the state space is two-dimensional, these failures are also easy to visualize and diagnose.

Spurious low-cost basins trap the agent. Visualizing the rollout trajectories reveals that in every failed episode, the agent becomes trapped in a local region for the remainder of the rollout. To see why the planner chooses such actions, we compute a cost map for each evaluated goal: we place the agent at every feasible position on a 4 px grid, render the observation, encode it with the LeWM encoder, and compute its L2 terminal cost against the goal latent (Equation 1). The top row of Figure 1 shows four examples overlaid with the executed rollouts. The cost always forms a basin around the goal, as expected: the cost is zero at the goal, and nearby observations yield similar latents. Only a short distance away, however, the cost rises sharply, while some regions far from the goal have a much lower cost than their surroundings. Consequently, when actions sampled by the CEM planner lead into one of these spurious basins rather than the true basin around the goal, the agent becomes trapped.

The L2 cost is faithful only locally. The simplicity of TwoRoom also provides a "ground-truth" cost for every position: the length of the shortest feasible path to the goal. Figure 2 plots the terminal cost against this path length for the rightmost pair in Figure 1. For positions within 12 px of the goal along the shortest path, the L2 cost orders positions faithfully (Spearman 0.91), consistent with the local basin around the goal in Figure 1. Beyond this radius, however, the cost varies widely and no longer increases with path length, dropping the overall Spearman correlation to -0.17.

Because the other three tasks have higher-dimensional state spaces, we instead measure how far a state is from the goal using a task-specific *physical distance* computed from the simulator states (Table 4; Appendix A.3). Across all four tasks, the median L2 cost increases with physical distance only over a short range before flattening out (Figure 4, top).

The latents retain the agent's position. Why does the L2 cost become uninformative away from the goal? Is it because the encoder fails to capture the task-relevant state, or simply because the cost

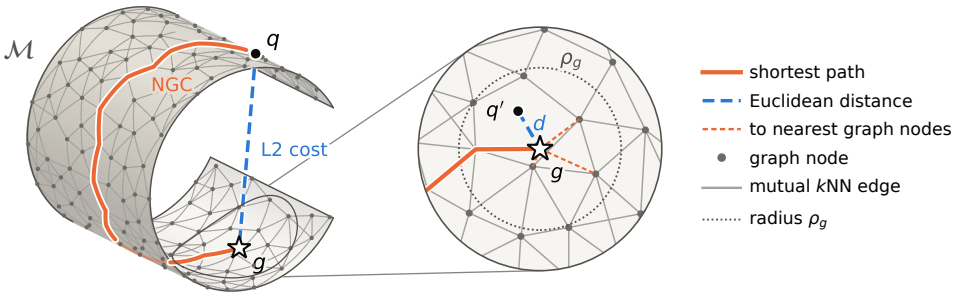


Figure 5: NGC on a curved manifold $\mathcal{M}$. **Left:** a candidate q is close to the goal g in Euclidean distance (blue, the L2 cost) but far from g along $\mathcal{M}$. NGC scores q by the shortest path through the graph (orange), whose edges join mutual 15-nearest neighbors. **Right:** the neighborhood of g, flattened. When a candidate q' and g lie within each other's neighborhood, NGC scores q' by their Euclidean distance $d = \|q' - g\|_2$. The surface is a schematic; the LeWM latents are 192-dimensional.

function fails to read it? Since a TwoRoom observation is determined by the agent's 2D position, latents that retain this information should admit a faithful two-dimensional embedding. Figure 3 applies five dimensionality reduction methods to the grid latents from Figure 1. Isomap (Tenenbaum et al., 2000), t-SNE (van der Maaten & Hinton, 2008), and UMAP (McInnes et al., 2018) construct embeddings from each latent's nearest neighbors, and all three unfold the two rooms and recover the physical layout up to a smooth deformation (Spearman 0.81, 0.83, and 0.92 between embedding distance and physical (x, y) distance). In contrast, principal component analysis (PCA; Pearson, 1901; Hotelling, 1933), which projects the latents linearly onto their two directions of largest variance, and metric multidimensional scaling (MDS; Borg & Groenen, 2005), which preserves pairwise Euclidean distances, both fold the room onto itself (Spearman 0.52 and 0.32). Thus, the latents do encode the underlying physical state required by the task, but recovering it requires respecting local neighborhoods rather than global Euclidean distances.

4 Hypothesis and a cost along the latent manifold

The observations in Section 3 lead us to the following hypothesis: the latents of a trained LeWM lie near a curved, low-dimensional manifold $\mathcal{M}$, and the terminal cost should measure the distance along $\mathcal{M}$ rather than through the ambient latent space.

This hypothesis offers a geometric interpretation of all four observations. Whereas the L2 cost measures the squared Euclidean distance between two latents in the ambient space (Equation 1), the distance along $\mathcal{M}$ is the length of the shortest path confined to the manifold. Within a sufficiently small, nearly flat neighborhood, the Euclidean distance closely approximates this path length, explaining why the L2 cost orders nearby states faithfully. Over longer distances, however, the manifold curves through the ambient space, so moving farther along $\mathcal{M}$ need not increase the Euclidean distance to the goal. This accounts for why the L2 cost ceases to increase with physical distance in Figure 2 and Figure 4 (top). Moreover, where the manifold curves back toward the goal latent, a region far away along $\mathcal{M}$ can have a lower L2 cost than its surroundings, creating the spurious basins observed in Figure 1 (top). Finally, because neighborhood-based embeddings rely on local relations, they can unfold the manifold and recover the underlying physical layout in Figure 3.

Because $\mathcal{M}$ has no explicit parameterization, we cannot compute distances along it analytically and instead estimate them from samples. On a neighborhood graph whose edges connect only nearby points, each edge approximates a short local segment of $\mathcal{M}$, so the shortest-path length between two nodes estimates the distance along $\mathcal{M}$. We use this estimator to define the **Neighborhood Graph Cost** (NGC, Figure 5). Offline, we encode a random subset of N training observations as graph nodes $v_1, \ldots, v_N$ and connect each pair of mutual 15-nearest neighbors with an edge weighted by their Euclidean distance $\|v_i - v_j\|_2$. We then precompute all-pairs shortest-path lengths D_{ij} on this between nodes v_i and v_j. Online, we attach the predicted terminal latent q of each candidate action

sequence and the goal latent g to their four nearest graph nodes, $\mathcal{N}_4(q)$ and $\mathcal{N}_4(g)$, and define the terminal cost as

$$C(q,g) = \begin{cases} \|q-g\|_2, & \|q-g\|_2 \le \min\{\rho_q, \rho_g\}, \\ \min\limits_{\substack{i\in\mathcal{N}_4(q)\\ j\in\mathcal{N}_4(g)}} \big[\|q-v_i\|_2 + D_{ij} + \|v_j-g\|_2\big], & \text{otherwise}, \end{cases} \tag{2}$$

where ρ_q and ρ_g denote the distances from q and g to their respective 15th-nearest graph nodes. When q and g lie within each other's local neighborhood, their Euclidean distance already approximates the manifold distance, so NGC uses it directly. NGC replaces only the terminal cost; the encoder and predictor remain frozen, and the planner is unchanged. Algorithm 1 in Appendix A.2 summarizes the procedure.

5 Experiments

We evaluate NGC by comparing the planning success rate of NGC against the L2 cost on the same trained LeWM. We first show that NGC improves success rates across all four tasks and that this improvement comes from measuring shortest-path distances on the graph. We then replace the learned predictor with an oracle predictor to disentangle cost errors from predictor errors. Finally, we evaluate the computational overhead of NGC and test whether increasing the CEM search budget allows the L2 cost to close the gap.

5.1 Experimental setup

We use the four tasks and datasets released with LeWM (Maes et al., 2026b;a): **TwoRoom**, a point agent navigating between two rooms through a doorway; **Reacher**, the two-link arm from the DeepMind Control Suite (Tassa et al., 2018); **Cube**, the single-cube manipulation task from OGBench (Park et al., 2025); and **PushT**, pushing a T-shaped block to a target pose (Chi et al., 2023). Following the official LeWM release, we train the world model for 10 epochs and adopt the default CEM planner settings (details in Appendix A.1).

Evaluation protocol. As in Maes et al. (2026b), each evaluation *pair* consists of a start and a goal observation drawn from the same training episode, where the goal is a fixed *offset* of steps after the start, and the agent must reach the goal within twice the offset in primitive steps. We sample pairs with offsets of 25, 50, and 100 (25, 50, and 75 on TwoRoom, where episodes have 101 steps). For each task and offset, we evaluate 300 fixed pairs divided into three sets of 100. Each set is drawn with its own pair seed and paired with an independently constructed graph, forming three full replicates of the evaluation. Within each set, every pair is evaluated across three planner seeds. We compare the two costs pair by pair: for each pair, we average the success rate over the three planner seeds and compute the difference between NGC and the L2 cost. We report the mean per-pair difference along with a 95% bootstrap confidence interval (Appendix A.4). Appendix A.3 provides the full protocol.

5.2 Latent planning performance

On TwoRoom and Reacher, the success rate of NGC is at least 99.4% at every offset, except 94.6% at the farthest on TwoRoom, whereas the L2 cost drops from 91% to 27% on TwoRoom and from 100% to 81% on Reacher (Figure 6). Replacing the L2 cost with distances along the latent manifold thus nearly solves both tasks. On Cube and PushT, NGC improves the success rate by 10 to 20 points at every offset except offset 25 on PushT (where both costs already exceed 90%), though both costs still struggle at the longest offsets. Section 5.4 examines how much of the remaining failure stems from the learned predictor.

The offset, however, is only a loose proxy for goal distance: the same number of expert steps can move the agent far or barely at all, mixing near and distant goals within each offset. To separate easy and hard goals more directly, the bottom row of Figure 6 regroups the evaluated pairs into quintiles of the physical distance between the start and goal states. From the nearest quintile to the farthest, the success rate of the L2 cost falls from 99.8% to 7.6% on TwoRoom and from 99.6% to 60% on Reacher. In contrast, NGC remains at or above 99.4% in every quintile on both tasks, except

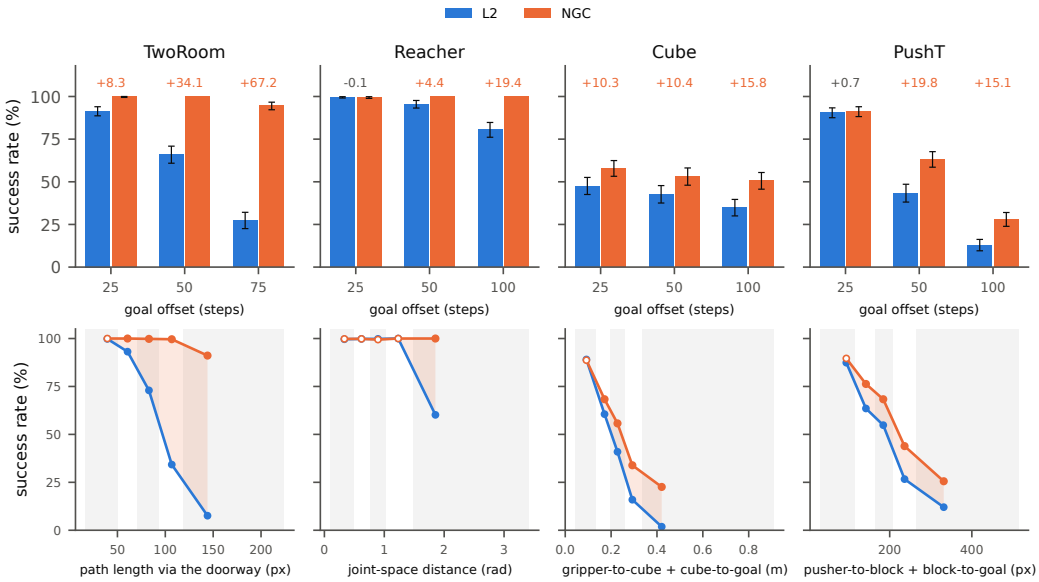


Figure 6: **Top:** Success rate against the goal offset (300 pairs per offset; whiskers indicate 95% bootstrap intervals over pairs). Numbers above the bars show the mean per-pair difference (NGC minus L2, in percentage points), highlighted in orange when the 95% interval excludes zero. **Bottom:** The same pairs pooled across offsets and binned into quintiles of the physical distance between the start and goal states (180 pairs per bin). Shaded regions show the gap between NGC and the L2 cost, and filled NGC markers indicate bins where the 95% interval of the difference excludes zero.

the farthest quintile on TwoRoom (91%), where the remaining failures arise from predictor errors (Section 5.4). On Cube and PushT, the gain is near zero in the nearest quintile and peaks in the farthest two, reaching 21 and 17 points. Across all tasks, the L2 cost degrades on physically distant goals, where NGC delivers its largest gains.

Revisiting the diagnostics of Section 3 reveals how NGC reshapes the cost landscape that the planner follows. Under NGC, the cost map of each TwoRoom pair forms a single clean basin around the goal (Figure 1, bottom), and the cost increases with path length across the entire room (Figure 2, bottom). Across all four tasks, NGC likewise increases monotonically with physical distance over its full range (Figure 4, bottom). Section 5.3 tests whether the planning gains indeed come from this graph-based distance estimate.

5.3 Ablation

Besides routing paths along the graph, NGC anchors the cost at graph nodes: the term $\|q - v_i\|_2$ in Equation 2 is at least the distance from q to its nearest graph node, growing as q moves away from the graph. This raises an alternative explanation for the gain: CEM might exploit predictor errors by selecting candidates whose predicted terminal latent q is close to g in Euclidean space but far from the training latents. Any cost that penalizes such out-of-distribution latents would prevent this exploitation, regardless of whether it measures distance along $\mathcal{M}$. To test this explanation, we evaluate *NGC with Euclidean node distances* (NGC_{Euc}), which replaces D_{ij} in Equation 2 with the direct Euclidean distance $\|v_i - v_j\|_2$ between the same graph nodes while keeping the rest of NGC unchanged. NGC_{Euc} applies the exact same anchor penalty to candidates far from the graph.

As shown in Table 1, NGC_{Euc} improves over the L2 cost only at offsets 50 and 75 on TwoRoom and offsets 50 and 100 on Reacher (by 1 to 14 points), while elsewhere remaining within 2 points of the L2 cost or falling 2 to 5 points below it. In contrast, replacing $\|v_i - v_j\|_2$ with shortest-path distances D_{ij} yields substantial gains across all tasks, with 95% confidence intervals excluding zero everywhere except offset 25 on Reacher (where the L2 cost already reaches 99.6%). Thus,

Table 1: Ablation of the graph shortest-path distance on the pairs and graphs of Figure 6. NGC_{Euc} replaces D_{ij} with the Euclidean distance between the same graph nodes. The last two columns report mean per-pair differences (in percentage points) with 95% bootstrap intervals.

Task	offset	L2	NGC_{Euc}	NGC	$\text{NGC}_{\text{Euc}} - \text{L2}$	$\text{NGC} - \text{NGC}_{\text{Euc}}$
TwoRoom	25	91.4	86.6	99.8	-4.9 $[-7.4, -2.3]$	$+13.2$ $[+10.0, +16.6]$
	50	65.9	69.0	100.0	$+3.1$ $[-1.4, +7.8]$	$+31.0$ $[+26.2, +36.0]$
	75	27.3	41.3	94.6	$+14.0$ $[+9.6, +18.6]$	$+53.2$ $[+48.0, +58.3]$
Reacher	25	99.6	99.3	99.4	-0.2 $[-0.9, +0.3]$	$+0.1$ $[-0.6, +0.8]$
	50	95.6	96.6	100.0	$+1.0$ $[+0.2, +2.0]$	$+3.4$ $[+1.7, +5.4]$
	100	80.6	82.2	100.0	$+1.7$ $[+0.2, +3.2]$	$+17.8$ $[+13.8, +22.0]$
Cube	25	47.6	44.1	57.9	-3.4 $[-6.0, -0.9]$	$+13.8$ $[+9.9, +17.7]$
	50	42.7	40.9	53.1	-1.8 $[-4.3, +0.7]$	$+12.2$ $[+8.4, +16.2]$
	100	34.8	33.0	50.6	-1.8 $[-4.6, +1.0]$	$+17.6$ $[+13.2, +22.0]$
PushT	25	90.6	88.2	91.2	-2.3 $[-4.8, 0.0]$	$+3.0$ $[+1.3, +4.9]$
	50	43.3	40.7	63.1	-2.7 $[-6.8, +1.4]$	$+22.4$ $[+17.8, +27.2]$
	100	12.8	9.8	27.9	-3.0 $[-5.3, -0.8]$	$+18.1$ $[+14.8, +21.7]$

Table 2: Success rate (%) with the learned predictor and with the oracle predictor, which executes each candidate in the simulator and encodes its terminal frame, on one set of 100 pairs with a single planner seed. The last two columns report mean per-pair differences in % with 95% bootstrap intervals: NGC minus L2 under the oracle predictor, and oracle minus learned predictor under NGC.

		L2		NGC		oracle	NGC
Task	offset	learned	oracle	learned	oracle	NGC − L2	oracle − learned
TwoRoom	25	92	84	100	100	$+16$ $[+9, +23]$	0
TwoRoom	50	65	58	100	100	$+42$ $[+32, +52]$	0
TwoRoom	75	24	19	94	100	$+81$ $[+73, +88]$	$+6$ $[+2, +11]$
Cube	25	44	56	59	76	$+20$ $[+11, +29]$	$+17$ $[+8, +26]$
Cube	50	44	52	53	71	$+19$ $[+11, +27]$	$+18$ $[+11, +26]$
Cube	100	31	38	50	74	$+36$ $[+26, +46]$	$+24$ $[+15, +33]$
PushT	25	90	94	88	95	$+1$ $[0, +3]$	$+7$ $[+2, +13]$
PushT	50	43	58	61	84	$+26$ $[+16, +36]$	$+23$ $[+14, +33]$
PushT	100	14	13	28	36	$+23$ $[+15, +32]$	$+8$ $[-2, +18]$

the improvement comes from measuring distance along the graph, which estimates the manifold distance, rather than from anchoring predicted latents near the training data.

5.4 Planning with an oracle predictor

All results so far use the learned predictor, whose rollout errors could affect both the gains of NGC and its remaining failures. To isolate the role of the terminal cost, we replace the learned predictor with an oracle predictor: for each CEM candidate, the simulator executes the action sequence from a copy of the exact environment state, and the frozen encoder encodes the resulting terminal frame. The encoder, graph, cost, and planner remain unchanged. Because querying the simulator for all 300 candidates across 30 CEM iterations is computationally expensive, we evaluate the oracle on one set of 100 pairs with a single planner seed, omitting Reacher where NGC already reaches 100% at the two longer offsets. Appendix A.5 provides further details.

If NGC merely compensated for errors of the learned predictor, the two costs would achieve similar success rates under the oracle. Instead, NGC still outperforms the L2 cost by 16 to 81 points in eight of the nine settings in Table 2. The gain of NGC therefore comes from correcting the terminal cost itself, not from compensating for an imperfect predictor. Notably, on TwoRoom, replacing the learned predictor with the oracle actually lowers the success rate of the L2 cost by 5 to 8 points (analyzed in Appendix E).

Table 3: Planning time of NGC at offset 25. CEM: median time of one CEM optimization, with its ratio to L2. Evaluation: total wall-clock time of a full evaluation relative to L2 on the same pairs. Objective: median time to score one pool of 300 candidates (Appendix A.2).

	CEM (s)		Evaluation	Objective (ms per pool)	
Task	L2	NGC	(× L2)	L2	NGC
TwoRoom	0.49	0.48 (0.98×)	0.86	0.071	0.67
Reacher	0.49	0.52 (1.06×)	1.05	0.070	0.66
Cube	0.49	0.53 (1.07×)	1.08	0.070	0.67
PushT	0.50	0.52 (1.05×)	1.13	0.070	0.67

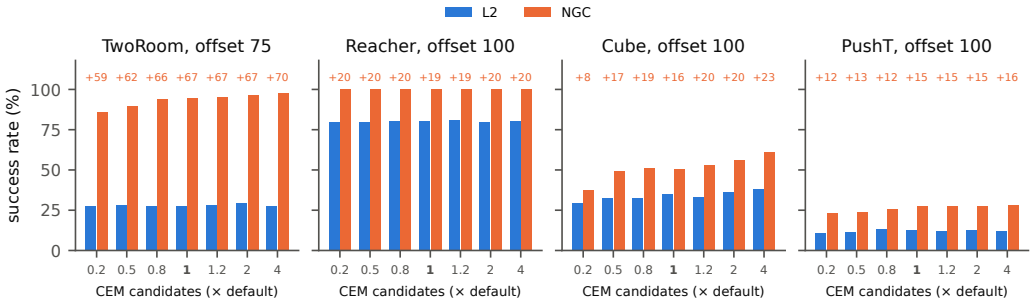


Figure 7: Success rate against the number of CEM candidates at the longest offset of each task. The number of elites scales proportionally; 1× corresponds to the default setting in Figure 6. The elite fraction and the 30 CEM iterations remain fixed, and all multipliers are evaluated on the pairs, planner seeds, and graphs of Figure 6.

The oracle predictor also shows how much of NGC's remaining failure stems from prediction errors, since such failures disappear once predictions are exact. On TwoRoom, prediction errors account for all remaining failures: paired with the oracle, NGC solves 100% of the pairs across all offsets, including the 6 pairs at offset 75 where rollouts with the learned predictor get stuck against the wall beside the doorway. On Cube and PushT, the oracle predictor raises the success rate of NGC by 7 to 24 points, showing that predictor errors also account for a substantial portion of the failures on these tasks. Even with exact predictions, however, NGC remains far from solving Cube and PushT, pointing to limitations beyond the predictor (Section 6).

5.5 Computation and planning time

Compared with the L2 cost, NGC introduces additional computation, but nearly all of it takes place offline. The shortest paths between all graph nodes are precomputed once after the model is trained, and attaching the goal latent to the graph is done once per episode. During planning, the online overhead of scoring candidates is small compared to predictor rollouts. Consequently, a single CEM optimization with NGC takes at most 7% longer than with the L2 cost, and a full evaluation takes 0.86× to 1.13× as long, running faster on TwoRoom because successful episodes terminate early (Table 3).

Conversely, giving the L2 cost more compute does not close the performance gap. Figure 7 scales the search budget by varying the number of CEM candidates from 0.2× to 4× the default at the longest offset of each task. Increasing the budget fourfold adds far more computation than NGC's online overhead, yet it improves the success rate of the L2 cost by no more than 3.2 points, leaving it well below default NGC on every task. What limits planning with the L2 cost is the cost geometry, not the search budget.

6 DISCUSSION AND CONCLUSION

Where does the manifold come from? LeWM is trained solely for next-latent prediction and regularization, without any supervision over state-to-state distances. To make latent distances plannable, many follow-ups therefore introduce explicit long-range supervision, such as temporal reachability or privileged physical state (Section 2). Yet without any such supervision, distances along the latent manifold of LeWM suffice to nearly solve TwoRoom and Reacher and yield consistent gains across all four tasks. Nearby states often produce similar observations and respond similarly to the same actions. We conjecture that predictive training preserves this local continuity while preventing representation collapse, naturally yielding latent neighborhoods that mirror the local structure of the state space.

Limitations. This local continuity hypothesis also points to a natural limitation at contact, where nearby physical states can respond very differently to the same action: a gripper a few millimeters away from a cube cannot lift it, whereas one grasping it can. If the encoder places such states in the same latent neighborhood, changing the terminal cost alone may not resolve the distinction. Indeed, NGC reaches near-perfect success on TwoRoom and Reacher, which involve smooth motion, but still falls short on the contact-rich Cube and PushT tasks even with an oracle predictor. Furthermore, contact introduces asymmetric transitions that an undirected graph cannot express. Finally, our evaluation is confined to the training distribution: following LeWM (Maes et al., 2026b), we draw start–goal pairs from episodes used to train the world model (though we exclude evaluated episodes from graph construction). Generalization to unseen environments or goals outside the coverage of the graph remains untested.

Future work. We plan to investigate how latent neighborhoods can better capture contact dynamics and the directionality of control. Another promising direction is to construct and evaluate NGC using data collected from random or autonomous exploration. Because NGC relies solely on observation latents without requiring actions or trajectory order, neither the graph nor the world model's prediction objective depends on expert demonstrations. Exploration data thus offers a natural next test of whether our findings extend beyond the demonstration datasets used here.

Conclusion. Many efforts to extend latent world models to longer horizons treat planning failures as a representation failure, retraining the encoder so that Euclidean distances measure task progress. Our results suggest that much of this failure lies not in what the representation encodes, but in how the planner reads it. Predictive training alone already yields faithful local neighborhoods, and a shortest-path graph over these neighborhoods turns them into an effective planning cost without any additional training. We therefore argue that latent world models should be built and evaluated around the geometry of their latent manifold, rather than expecting global Euclidean distances in the ambient space to measure progress directly.

AI USE STATEMENT

In this work, we used LLM-based assistants to search for and summarize related literature, implement experiments, create visualizations, and translate the authors' intention to academic English writing. We have reviewed all AI-assisted work. Every number in the paper is copied from archived experiment reports, and every figure is generated by a script from those records. Every citation was checked against the cited work. We are aware that AI-generated text can be painful to read, and we have tried our best to ensure that the presentation is clear, accurate, and readable to human readers.

ETHICS STATEMENT

This work uses only simulated benchmark environments and publicly released datasets and checkpoints. It involves no human subjects, personal data, or deployment on physical systems. We see no direct ethical concerns beyond those common to robot-learning research.

REPRODUCIBILITY STATEMENT

All experiments use the public `stable-worldmodel` package and the released LeWM datasets. Section 5.1 and Appendix A specify the checkpoints, the fixed pair sets, the graph construction

rule, the planner budget, the numerical settings, and the statistical procedure. Every reported cell is backed by a per-pair outcome record. The code is available at https://github.com/youzunzhi/ngc-latent-manifold.

REFERENCES

Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025. URL https://arxiv.org/abs/2506.09985.

Jiaxin Bai and Jiaxuan Xiong. Temporal-distance JEPA: Plan-aware representation learning for latent world model predictive control. *arXiv preprint arXiv:2607.25337*, 2026. URL https://arxiv.org/abs/2607.25337.

Ingwer Borg and Patrick J. F. Groenen. *Modern Multidimensional Scaling: Theory and Applications*. Springer Series in Statistics. Springer, 2 edition, 2005. ISBN 978-0-387-25150-9. doi: 10.1007/0-387-28981-X. URL https://link.springer.com/book/10.1007/0-387-28981-X.

Niccolò Caselli, Francesco Massafra, Samuele Punzo, Salvatore Lo Sardo, Ippokratis Pantelidis, and Sathya Kamesh Bhethanabhotla. Mind the gap: Promises and pitfalls of hierarchical planning in LeWorldModel. *arXiv preprint arXiv:2607.12547*, 2026. URL https://arxiv.org/abs/2607.12547.

Letian Cheng, Qi Zhang, and Yisen Wang. SAGE: Subgoal-conditioned action generation for latent world model planning. *arXiv preprint arXiv:2607.17973*, 2026. URL https://arxiv.org/abs/2607.17973.

Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin CM Burchfiel, and Shuran Song. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. In *Proceedings of Robotics: Science and Systems*, Daegu, Republic of Korea, July 2023. doi: 10.15607/RSS.2023.XIX.026. URL https://roboticsproceedings.org/rss19/p026.html.

Pieter-Tjerk de Boer, Dirk P. Kroese, Shie Mannor, and Reuven Y. Rubinstein. A tutorial on the cross-entropy method. *Annals of Operations Research*, 134(1):19–67, 2005. doi: 10.1007/s10479-005-5724-z. URL https://link.springer.com/article/10.1007/s10479-005-5724-z.

Matthieu Destrade, Oumayma Bounou, Quentin Le Lidec, Jean Ponce, and Yann LeCun. Value-guided action planning with JEPA world models. In *The World Modeling Workshop*, Montreal, QC, Canada, February 2026. URL https://arxiv.org/abs/2601.00844. Poster.

Frederik Ebert, Chelsea Finn, Sudeep Dasari, Annie Xie, Alex Lee, and Sergey Levine. Visual foresight: Model-based deep reinforcement learning for vision-based robotic control. *arXiv preprint arXiv:1812.00568*, 2018. URL https://arxiv.org/abs/1812.00568.

Benjamin Eysenbach, Ruslan Salakhutdinov, and Sergey Levine. Search on the replay buffer: Bridging planning and reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32. Curran Associates, Inc., 2019. URL https://papers.nips.cc/paper_files/paper/2019/hash/5c48ff18e0a47baaf81d8b8ea51eec92-Abstract.html. arXiv:1906.05253.

Chelsea Finn and Sergey Levine. Deep visual foresight for planning robot motion. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 2786–2793. IEEE, May 2017. doi: 10.1109/ICRA.2017.7989324.

Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In Kamalika Chaudhuri and Ruslan Salakhutdinov (eds.), *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 2555–2565. PMLR, 09–15 Jun 2019. URL https://proceedings.mlr.press/v97/hafner19a.html.

Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations*, pp. 47376–47405, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/cf73d57b6dcda32b293df7c2d5341f49-Abstract-Conference.html.

Harold Hotelling. Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6):417–441, 1933. doi: 10.1037/h0071325. URL https://psycnet.apa.org/doi/10.1037/h0071325.

Jiaming Hu, Yan Zheng, and Tian Wang. SCALE: State-calibrated latent embeddings for JEPA planning in the right geometry. *arXiv preprint arXiv:2608.16287*, 2026. URL https://arxiv.org/abs/2608.16287.

Xiaodi Huang, Ziyi Ding, Jingtian Wan, Yuchen Liu, Yuan Zhang, Xiao-Ping Zhang, Jiayu Chen, Zhang Zhang, and Tao Huang. Traj-LeWM: Path-aware world-model planning via latent trajectory cost. *arXiv preprint arXiv:2608.14125*, 2026. URL https://arxiv.org/abs/2608.14125.

Yann LeCun. A path towards autonomous machine intelligence. OpenReview preprint, 2022. URL https://openreview.net/forum?id=BZ5a1r-kVsf. Version 0.9.2.

Liangyu Li, Shengzhi Wang, Libin Qiu, Mingliang Xiong, and Qingwen Liu. World model control by trajectory reachability metrics. *arXiv preprint arXiv:2605.22164*, 2026a. URL https://arxiv.org/abs/2605.22164.

Wenyuan Li, Guang Li, Keisuke Maeda, Takahiro Ogawa, and Miki Haseyama. Predictive but not plannable: RC-aux for latent world models. *arXiv preprint arXiv:2605.07278*, 2026b. URL https://arxiv.org/abs/2605.07278.

Zihan Liu, Yuzhe Zhuang, Yuanzu Li, Wanshuang Gou, Jiahong Liu, Min Zhou, and Menglin Yang. ProWorld: Progress-aware hyperbolic world models for long-horizon visual goal reaching. *arXiv preprint arXiv:2608.01926*, 2026. URL https://arxiv.org/abs/2608.01926.

Lucas Maes, Quentin Le Lidec, Luiz Facury, Nassim Massaudi, Ayush Chaurasia, Francesco Capuano, Richard Gao, Taj Gillin, Dan Haramati, Damien Scieur, Yann LeCun, and Randall Balestriero. stable-worldmodel: A platform for reproducible world modeling research and evaluation. *arXiv preprint arXiv:2605.21800*, 2026a. URL https://arxiv.org/abs/2605.21800.

Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026b. URL https://arxiv.org/abs/2603.19312.

Sergi Masip, Jonathan Swinnen, Yutong Hu, Renaud Detry, and Tinne Tuytelaars. FF-JEPA: Long-horizon planning in world models with latent planners. *arXiv preprint arXiv:2606.09311*, 2026. URL https://arxiv.org/abs/2606.09311.

Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018. URL https://arxiv.org/abs/1802.03426.

Seohong Park, Kevin Frans, Benjamin Eysenbach, and Sergey Levine. OGBench: Benchmarking offline goal-conditioned RL. In Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu (eds.), *International Conference on Learning Representations*, volume 2025, pp. 94937–94982, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/ecd92623ac899357312aaa8915853699-Abstract-Conference.html.

Karl Pearson. On lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11):559–572, 1901. doi: 10.1080/14786440109462720. URL https://www.tandfonline.com/doi/abs/10.1080/14786440109462720.

Phu Pham and Aniket Bera. Latent energy action planning with world models. *arXiv preprint arXiv:2609.03294*, 2026. URL https://arxiv.org/abs/2609.03294.

Reuven Rubinstein. The cross-entropy method for combinatorial and continuous optimization. *Methodology and Computing in Applied Probability*, 1(2):127–190, 1999. doi: 10.1023/A:1010091220143. URL https://link.springer.com/article/10.1023/A:1010091220143.

Nikolay Savinov, Alexey Dosovitskiy, and Vladlen Koltun. Semi-parametric topological memory for navigation. In *International Conference on Learning Representations (ICLR)*, 2018. URL https://github.com/nsavinov/SPTM. arXiv:1803.00653.

Joyjeet Singh. The objective is the bottleneck: Latent world models encode what their planners cannot use. *arXiv preprint arXiv:2608.12959*, 2026. URL https://arxiv.org/abs/2608.12959.

Uladzislau Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Learning from reward-free offline data: A case for planning with latent dynamics models. In *Advances in Neural Information Processing Systems*, volume 38, pp. 43905–43941. Curran Associates, Inc., 2025. doi: 10.52202/085713-1465. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/3e7cf447f21cd11c846463affefce665-Abstract-Conference.html.

Armin Sommer and Jannik Schilling. Reinforced planning with latent world models. *arXiv preprint arXiv:2608.18669*, 2026. URL https://arxiv.org/abs/2608.18669.

Junhan Sun, Hao Zhao, and Guofeng Zhang. INTACT: Isomorphic intent-to-action learning for search-free world models. *arXiv preprint arXiv:2607.26056*, 2026. URL https://arxiv.org/abs/2607.26056.

Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, Timothy Lillicrap, and Martin Riedmiller. DeepMind control suite. *arXiv preprint arXiv:1801.00690*, 2018. URL https://arxiv.org/abs/1801.00690.

Joshua B. Tenenbaum, Vin de Silva, and John C. Langford. A global geometric framework for nonlinear dimensionality reduction. *Science*, 290(5500):2319–2323, 2000. doi: 10.1126/science.290.5500.2319. URL https://www.science.org/doi/10.1126/science.290.5500.2319.

Basile Terver, Tsung-Yen Yang, Jean Ponce, Adrien Bardes, and Yann LeCun. What drives success in physical planning with joint-embedding predictive world models? *Transactions on Machine Learning Research*, May 2026. URL https://openreview.net/forum?id=cHZn5Gdh8e.

Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9(86):2579–2605, 2008. URL https://www.jmlr.org/papers/v9/vandermaaten08a.html.

Jiawei Wang, Ke Rui, Yushen Zuo, Yichun Feng, and Minglei Li. Decision-metric alignment in latent world models: Diagnostics and action-conditioned objectives for MPC planning. *arXiv preprint arXiv:2608.18746*, 2026a. URL https://arxiv.org/abs/2608.18746.

Ying Wang, Oumayma Bounou, Gaoyue Zhou, Randall Balestriero, Tim G. J. Rudner, Yann LeCun, and Mengye Ren. Temporal straightening for latent planning. *arXiv preprint arXiv:2603.12231*, 2026b. URL https://arxiv.org/abs/2603.12231.

Ge Yang, Amy Zhang, Ari Morcos, Joelle Pineau, Pieter Abbeel, and Roberto Calandra. Plan2Vec: Unsupervised representation learning by latent plans. In Alexandre M. Bayen, Ali Jadbabaie, George Pappas, Pablo A. Parrilo, Benjamin Recht, Claire Tomlin, and Melanie Zeilinger (eds.), *Proceedings of the 2nd Conference on Learning for Dynamics and Control*, volume 120 of *Proceedings of Machine Learning Research*, pp. 935–946. PMLR, 10–11 Jun 2020. URL https://proceedings.mlr.press/v120/yang20b.html.

Lunjun Zhang, Ge Yang, and Bradly C Stadie. World model as a graph: Learning latent landmarks for planning. In Marina Meila and Tong Zhang (eds.), *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pp. 12611–12620. PMLR, 18–24 Jul 2021. URL https://proceedings.mlr.press/v139/zhang21x.html.

Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (eds.), *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 79115–79135. PMLR, 13–19 Jul 2025. URL https://proceedings.mlr.press/v267/zhou25t.html.

Algorithm 1 Neighborhood Graph Cost (NGC)

Offline, once per encoder

Require: a random subset of N training observations $o_1, \dots, o_N$

$v_i \leftarrow E(o_i)$ for $i = 1, \dots, N$ ▷ graph nodes

join i and j by an edge of length $\|v_i - v_j\|_2$ if each is among the 15 nearest graph nodes of the other

while the graph has more than one component **do**

  $(i, j) \leftarrow \arg\min_{c(i) \neq c(j)} \|v_i - v_j\|_2$ ▷ $c(i)$: component of v_i

  join i and j by an edge of length $\|v_i - v_j\|_2$ ▷ merges $c(i)$ and $c(j)$

end while

$D_{ij} \leftarrow$ shortest-path length between i and j for all i, j ▷ all-pairs shortest paths via Dijkstra

Online, once per goal

Require: goal latent g

$\mathcal{N}_4(g) \leftarrow$ the 4 nearest graph nodes of g; $\rho_g \leftarrow$ distance from g to its 15th-nearest graph node

Online, for each candidate scored by CEM

Require: predicted terminal latent q

$\mathcal{N}_4(q) \leftarrow$ the 4 nearest graph nodes of q; $\rho_q \leftarrow$ distance from q to its 15th-nearest graph node

$d \leftarrow \|q - g\|_2$

if $d \leq \min\{\rho_q, \rho_g\}$ **then return** d ▷ Equation 2, first case

end if

return $\min_{i \in \mathcal{N}_4(q),\, j \in \mathcal{N}_4(g)} \left[\|q - v_i\|_2 + D_{ij} + \|v_j - g\|_2\right]$ ▷ Equation 2, second case

A Implementation and experimental details

A.1 World model and planner

World model. The world models we train follow the released LeWM recipe. The encoder is a ViT-tiny (patch size 14, image size 224, width 192) trained from scratch. The predictor is a six-layer transformer (16 heads, MLP width 2048) conditioned on a history of three frames. The action embedder has hidden width 768, the two projector heads have hidden width 2048, and SIGReg has weight 0.09 (17 knots, 1024 projections). Training uses AdamW (learning rate 5×10^{-5} with a linear warmup over the first 1% of steps and cosine decay, weight decay 10^{-3}), batch size 128, bf16 precision, gradient clipping at 1.0, and frameskip 5. Every task is trained for 10 epochs, over which the learning-rate schedule completes. Unless stated otherwise, we use the epoch-10 checkpoint (our checkpoint). We also evaluate the checkpoints released by Maes et al. (2026b) on HuggingFace (`quentinll/lewm-*`) to verify that our evaluator reproduces their published success rates (Appendix C.1) and to test whether the gains of NGC generalize across checkpoints (Appendix C.2).

Planner. CEM uses identical settings across all tasks. A plan spans $H = 5$ predictor steps of five primitive actions each (25 primitive steps). Each of the 30 iterations samples 300 candidate action sequences and refits the sampling distribution to the 30 elites with the lowest terminal cost. The planner executes all five blocks of the final elite mean before running the next CEM optimization from the resulting state. An episode terminates at the first primitive step that meets the success criterion, or after twice the offset in primitive steps.

A.2 NGC and its graph

For each task and pair seed, the graph nodes consist of $N = 5{,}000$ training observations drawn from distinct episodes that are excluded from evaluation under that seed. Each node is a single random frame from its episode, and every checkpoint encodes the exact same set of observations. If the mutual-15NN graph of a checkpoint has multiple connected components, we iteratively join them via their shortest inter-component edges (Algorithm 1), leaving all other edges unchanged.

Computing the cost. Our implementation stores the node latents and the all-pairs shortest-path table D in single precision on the planning device. For $N = 5{,}000$, this requires 103.8 MB of

memory: $N^2 \times 4\,\mathrm{B} = 100\,\mathrm{MB}$ for D and $N \times 192 \times 4\,\mathrm{B} = 3.8\,\mathrm{MB}$ for the latents. Because the goal's nearest graph nodes, their distances to the goal, and ρ_g remain fixed throughout an episode, they are computed once per goal. Scoring a candidate online then requires only a single nearest-neighbor query and taking the minimum over the 16 anchor pairs in Equation 2.

Timing. Table 3 benchmarks this computation on a single RTX 4090 GPU. The objective time is measured over 200 pools of 300 candidates per graph and averaged across the three graphs of each task. The CEM and evaluation times correspond to the offset-25 evaluations in Table 5. Because total evaluation time sums across all CEM optimizations in an evaluation, it also reflects how many planning steps episodes require before terminating. NGC runs fewer optimizations than the L2 cost on TwoRoom (938 versus 1,070) and Cube (1,341 versus 1,406) because it solves more episodes early, while running the same number of optimizations on Reacher and PushT.

A.3 Evaluation protocol

Seeds. Three types of random seeds structure our evaluation. A *pair seed* draws the evaluation pairs for a task (one set of 100 pairs per offset, yielding 300 pairs in total across three pair seeds) and samples the nodes of the corresponding NGC graph; the three pair seeds thus form three full replicates of the evaluation. A *planner seed* controls the CEM sampling distribution; every pair is evaluated across three planner seeds, which serve as repeated measurements and are averaged within each pair (Appendix A.4). Finally, a *training seed* controls the world model training run. All evaluation pair sets were drawn after NGC and the evaluation protocol were finalized, independently of the pairs used during method development.

Pair sets. Each task, offset, and pair seed defines a fixed set of 100 pairs drawn from 100 distinct episodes. A candidate pair consists of any start step in a dataset episode and the goal observation located one offset later in the same episode. Following Sun et al. (2026), we discard candidates whose start state already satisfies the goal's success criterion, and on TwoRoom we additionally discard candidates whose start and goal lie in the same room. We sample 100 distinct episodes uniformly from those retaining at least one valid candidate pair after filtering, and then sample one valid start step uniformly within each selected episode.

Execution and success. Each episode initializes the simulator to the exact state of the start observation, except that all velocities are zeroed out. Because the planner observes only the single start image, which conveys pose but not velocity, a nonzero initial velocity would cause unobservable drift that the planner cannot anticipate. The first planning step uses this single-frame context, while subsequent CEM optimizations use the executed history of three frames at stride 5, matching the training configuration. Table 4 lists the task-specific success criteria. On Reacher, we compare the shoulder angle on the unit circle as proposed by Sun et al. (2026): because the shoulder is an unlimited hinge, angles separated by a full rotation represent the exact same physical configuration and render the same image. Appendix C.1 also evaluates the unmodified LeWM protocol for direct comparison with published results.

Physical distance. To measure how far a state is from the goal in the environment, we compute a task-specific *physical distance* from simulator states (Table 4). On TwoRoom, this is the shortest feasible path length via the doorway; on Reacher, it is the Euclidean distance in joint space, so both directly measure the remaining motion. On Cube and PushT, the metric approximates the remaining work: it sums the end-effector-to-object and object-to-goal distances, without accounting for grasping dynamics or the detour required to position the pusher behind the block. In the bottom row of Figure 6, we bin evaluation pairs by the physical distance between their start and goal states, as a fixed number of expert steps can correspond to either large or negligible physical displacement. We also use this metric in Appendix B.2 to evaluate how faithfully each terminal cost orders states.

A.4 Statistics

Each pair is evaluated under both costs using the same three planner seeds. Averaging success over the three seeds yields a per-pair success rate for each cost, and their difference (NGC minus the L2 cost) takes a value in $\{0, \pm\frac{1}{3}, \pm\frac{2}{3}, \pm 1\}$ (Figure 9). We denote the mean per-pair difference (in %) by

Table 4: Success criteria and physical distances (Appendix A.3). An episode succeeds at the first step that meets the criterion.

Task	Success criterion	Physical distance
TwoRoom	Endpoint within 16 px of the goal position	Path length via the doorway
Reacher	Joint error < 0.05 rad, shoulder angle compared on the circle	Norm of the difference of the two joint angles, shoulder angle compared on the circle
Cube	Cube center within 4 cm of the goal position	Gripper-to-cube plus cube-to-goal distance
PushT	Pusher and block within 20 px of the goal pose and block angle within 20°	Pusher-to-block distance plus the root-mean-square displacement over the block's area to the goal pose

Δ. Its 95% confidence interval estimates how much Δ would vary across new draws of evaluation pairs. We compute this interval via a paired bootstrap over the evaluated pairs: a *resample* draws, with replacement, as many pairs as were evaluated in the comparison (300 in Table 5), and we take the 2.5th and 97.5th percentiles of Δ across $50{,}000$ such resamples. The interval reflects variation in the per-pair differences rather than in either cost's absolute success rate. Pairs on which both costs perform identically contribute zero, while the others contribute positive or negative differences (the colored segments in Figure 9). The interval width depends on the distribution of these differences, including the proportion of zeros. The interval reflects the variation across evaluation pairs for the evaluated checkpoint, graphs, and planner seeds; Tables 8, 9, and 10 report the variation across training seeds and graph construction. Intervals are not corrected for multiple comparisons. Where every pair has the same value, such as an arm that succeeds on every pair, all resamples coincide and we report no interval. The physical-distance bins of Figure 6 pool the offsets and can hold several pairs of one episode, so their resamples draw episodes instead of pairs.

A.5 Oracle predictor

The oracle predictor in Section 5.4 replaces only the learned predictor inside CEM. To score a candidate action sequence, it copies the current simulator state of the ongoing episode into a secondary simulator instance, steps the candidate forward, renders the terminal frame, and encodes it with the frozen encoder. The exact state copied depends on the environment: on TwoRoom, it includes the agent and target positions, wall and doorway parameters, and rendering variation variables; on Cube, it is the full MuJoCo physics data structure. On PushT, directly setting the state does not restore the internal state of the physics solver, so the oracle instead resets the secondary simulator and replays all actions executed in the episode so far. To match the fixed rollout horizon of the learned predictor, the secondary simulator executes each candidate action sequence in full, even if the agent passes through the goal state midway. The scored terminal frame thus always corresponds to the state after the final action of the candidate sequence. We run the oracle evaluation on the pair sets of the first pair seed, using its corresponding graph and a single planner seed; the learned-predictor columns of Table 2 are reported on the exact same pairs and planner seed.

B More details of the cost diagnostics

B.1 TwoRoom diagnostics

Cost maps. The cost maps in Figure 1 evaluate $C(E(\mathrm{render}(x, y)), E(o_{\mathrm{goal}}))$ using our checkpoint on a 46×46 grid with a 4 px stride, restricted to the $1{,}882$ collision-free positions where the simulator allows the agent center to be placed. Each image pixel is assigned the cost of its nearest grid point (at most 2 px away). Each panel is normalized independently to its own 2nd–98th percentile range, so color scales are comparable only within a single panel. Dark gray indicates the walls, light gray marks valid grid positions that the environment dynamics cannot reach, and the white trajectory curve connects the agent positions recorded after each planner step. Both rows are taken from the offset-25 evaluation on the first pair set (the same set used for the oracle experiments), where the L2 cost solves 92 pairs and NGC solves all 100. The four columns show one pair where the L2 cost succeeds and three of the 8 pairs where it fails. In these three failures, the L2 rollout

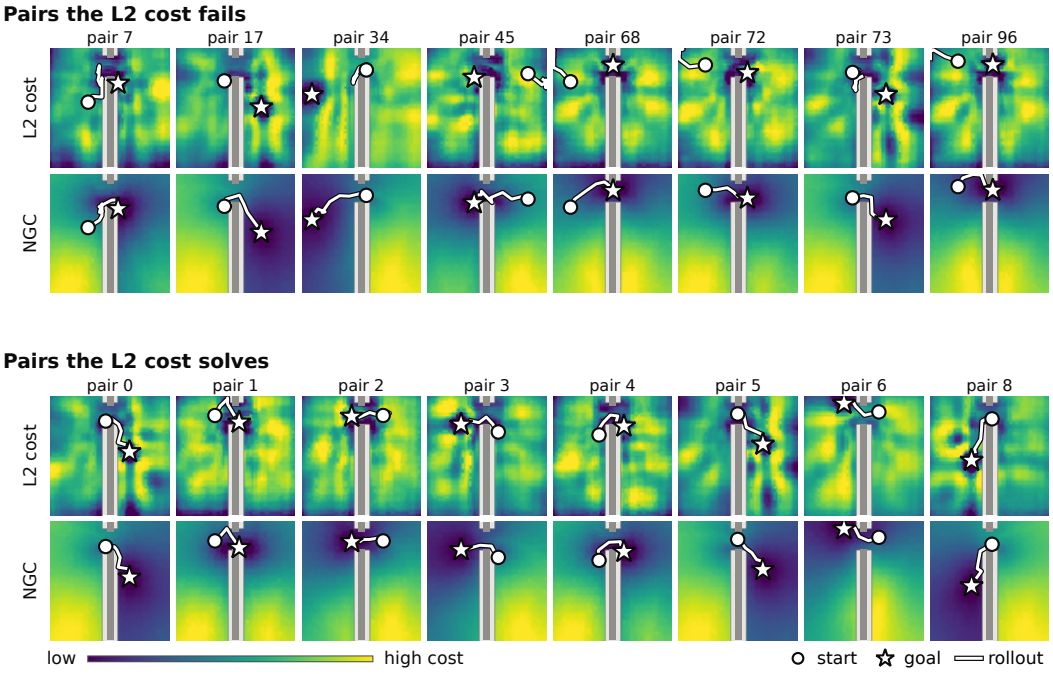


Figure 8: Cost maps and rollout trajectories for additional TwoRoom pairs, formatted as in Figure 1, with the L2 cost map above the NGC map of each pair. **Upper block:** all 8 pairs from the evaluation of Figure 1 on which the L2 cost fails. **Lower block:** the first 8 pairs in pair-index order on which the L2 cost succeeds. NGC solves every pair shown.

terminates in the cheapest 5%, 30%, and 3% of the cost map, despite remaining 93, 109, and 83 px away from the goal along the shortest path (trapped in a low-cost band along a far corner of the start room, against its far wall, and in a pocket beside the doorway, respectively). Figure 8 shows cost maps for all 8 failures of the L2 cost in this evaluation alongside the first 8 successes in pair-index order.

Projections. Figure 3 shows two-dimensional embeddings of the 1,882 grid latents produced by five dimensionality reduction methods. Each embedding is fit once on the grid latents and plotted directly. The first two methods operate on pairwise Euclidean distances: PCA projects the unwhitened latents onto their top two principal components, while metric MDS minimizes the raw stress $\sum_{i<j}(\|y_i - y_j\| - \|z_i - z_j\|)^2$ via SMACOF initialized from the classical MDS solution, reaching a Kruskal stress-1 of 0.375. The remaining three methods build on local nearest neighbors: Isomap uses an undirected 8-nearest-neighbor union graph followed by classical MDS; t-SNE uses perplexity 50, PCA initialization, and a fixed random seed; and UMAP uses 15 neighbors, a minimum distance of 0.1, and a fixed random seed.

B.2 Cost and physical distance

On tasks where the state space cannot be exhaustively gridded as in TwoRoom (Figure 2), Figure 4 uses the task-specific physical distance (Appendix A.3) as a reference to test how well each cost orders states by their distance to the goal. For each task, we sample 10,000 frames uniformly from dataset episodes not used in graph construction and evaluate all $10{,}000 \times 9{,}999 \approx 10^8$ ordered frame–goal pairs (treating the first frame as the state and the second as the goal). Because physical distances are computed directly from the underlying simulator states, this diagnostic is purely offline and involves no planning. Pairs are grouped into equal-width bins spanning from zero to the maximum observed physical distance; bins containing fewer than 1,000 pairs (accounting for fewer

than 700 pairs in total per task) are omitted. Across the other two pair seeds, the resulting Spearman correlations remain within 0.03 of those reported in Figure 4.

C More results on planning performance

Table 5 reports the numerical success rates corresponding to Figure 6, and Figure 9 breaks down the per-pair differences underlying each Δ (Appendix A.4). In the following subsections, we first benchmark our evaluation pipeline against published results under the original LeWM protocol, and then evaluate NGC across different model checkpoints and training seeds on the pairs of Figure 6.

Table 5: Success rate (%) of the L2 cost and NGC across the offsets of Figure 6 (our checkpoint, 300 pairs per offset, three planner seeds averaged within each pair). Brackets in small type give the 95% bootstrap confidence interval over pairs (corresponding to the whiskers in Figure 6). Δ is the mean per-pair difference (NGC minus L2) with its 95% confidence interval, and gained/lost counts the pairs with a positive versus negative difference (Figure 9).

Task	Offset	L2	NGC	Δ [95% CI]	gained/lost
TwoRoom	25	91.4 [88.7, 94.0]	99.8 [99.4, 100.0]	$+8.3$ [$+5.8$, $+11.0$]	45/1
TwoRoom	50	65.9 [60.9, 70.9]	100.0	$+34.1$ [$+29.2$, $+39.2$]	121/0
TwoRoom	75	27.3 [22.6, 32.1]	94.6 [92.2, 96.7]	$+67.2$ [$+62.2$, $+72.1$]	223/0
Reacher	25	99.6 [99.1, 99.9]	99.4 [98.9, 99.9]	-0.1 [-0.8, $+0.6$]	4/5
Reacher	50	95.6 [93.2, 97.7]	100.0	$+4.4$ [$+2.3$, $+6.9$]	15/0
Reacher	100	80.6 [76.1, 84.8]	100.0	$+19.4$ [$+15.2$, $+23.8$]	68/0
Cube	25	47.6 [42.6, 52.6]	57.9 [53.2, 62.4]	$+10.3$ [$+6.3$, $+14.4$]	81/44
Cube	50	42.7 [37.6, 47.8]	53.1 [48.0, 58.1]	$+10.4$ [$+6.8$, $+14.2$]	70/26
Cube	100	34.8 [30.0, 39.7]	50.6 [45.7, 55.4]	$+15.8$ [$+11.6$, $+20.0$]	96/26
PushT	25	90.6 [87.6, 93.3]	91.2 [88.2, 94.0]	$+0.7$ [-1.0, $+2.4$]	16/13
PushT	50	43.3 [38.1, 48.6]	63.1 [58.6, 67.7]	$+19.8$ [$+15.1$, $+24.4$]	115/32
PushT	100	12.8 [9.6, 16.2]	27.9 [23.9, 32.0]	$+15.1$ [$+11.6$, $+18.8$]	103/19

C.1 Reproducing the LeWM protocol

To compare directly with published numbers, we also replicate the exact random pair-sampling protocol of Maes et al. (2026b). This protocol consists of three runs, each drawing 50 start frames at random from the dataset episodes and taking the frame 25 steps later in the same episode as the goal (150 distinct pairs in total, each evaluated with a single planner seed). Execution and success criteria also follow the original LeWM setup; in particular, each episode initializes the simulator with its recorded nonzero velocity rather than zeroing it out (cf. Appendix A.3). Our planner configuration differs only in the number of CEM iterations: Maes et al. (2026b) report 10 iterations on TwoRoom, Reacher, and Cube and 30 on PushT, whereas we run 30 iterations across all tasks, matching their released evaluation code. Episodes selected for evaluation are excluded from graph construction.

Table 6 compares our L2 baseline against the numbers reported by Maes et al. (2026b). Using their released checkpoint, our L2 success rates lie 2 to 7 points below the published figures: the difference is within one standard deviation across the three runs on TwoRoom, Reacher, and Cube, and slightly larger on PushT (89.3% versus 96%). An independent re-evaluation of LeWM by Li et al. (2026b) similarly obtains 90.4% on PushT and remains within 4 points of our released-checkpoint L2 numbers across all tasks, showing that a similar PushT discrepancy also occurs in an independent evaluation. Using our own checkpoint, the L2 cost stays within 6 points of the published numbers on every task.

Crucially, the gains of NGC do not depend on the protocol refinements introduced in Appendix A.3. Under the original LeWM protocol, NGC keeps its gain on TwoRoom with both checkpoints and on Cube with our checkpoint, with 95% intervals excluding zero, and stays within 3 points of the L2 cost on Reacher and PushT. On Reacher, the L2 cost with our checkpoint drops from 99.6% under our protocol (Table 5) to 86.0% under the LeWM protocol. Two features of the LeWM protocol can contribute to failures: the initial velocity is not observable from the start image, and comparing

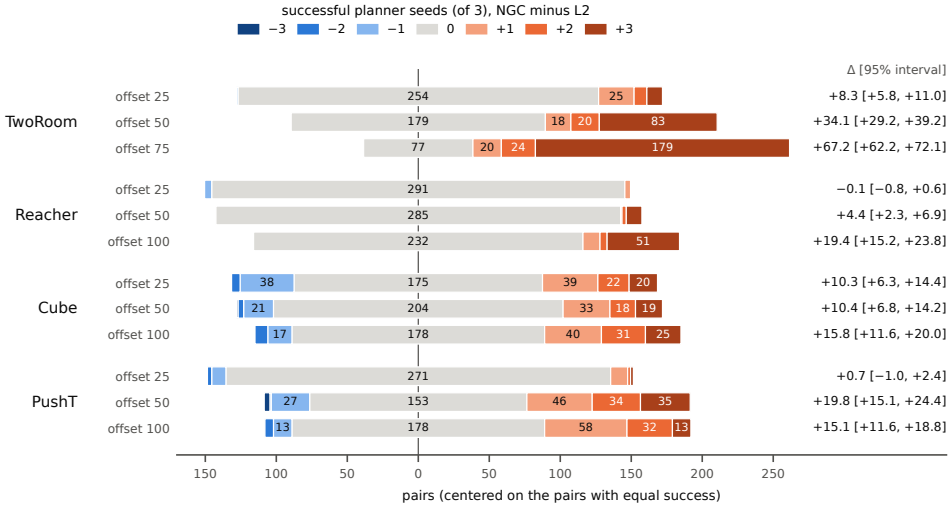


Figure 9: Per-pair differences underlying Table 5. Each bar represents the 300 pairs for a given task and offset. Under each cost, every pair is evaluated with three planner seeds; colors indicate the number of seeds on which NGC succeeds minus the number on which the L2 cost succeeds. Gray segments (equal success under both costs) are centered at zero; pairs where NGC improves over L2 extend to the right in orange, and pairs where it falls behind extend to the left in blue. Numbers count the pairs in each segment (segments with fewer than 13 pairs are unlabeled). **Right:** Δ, the mean per-pair difference in % (where one seed corresponds to $\frac{1}{3}$, or 33.3 points, of a single pair's score), with its 95% confidence interval.

raw shoulder angles treats identical poses separated by a 2π rotation as failures. Because the pair sets and other protocol settings also differ, this comparison does not isolate the contribution of either feature. On Cube with the released checkpoint, NGC gains less under the LeWM protocol ($+6.0$) than at offset 25 of our protocol ($+11.2$, Table 7) because random sampling retains pairs that already satisfy the success criterion at the start state (38% of sampled Cube pairs; Sun et al., 2026), which trivially succeed under both costs.

Table 6: Evaluation under the original LeWM protocol. Success (%) is reported as the mean $\pm$ SD across three runs of 50 random pairs each. The last column lists the integer-rounded figures reported by Maes et al. (2026b) (their Figure 6). Δ is the mean per-pair difference across all 150 pairs (NGC minus L2) with its 95% confidence interval (Appendix A.4).

	our checkpoint			released LeWM checkpoint			
Task	L2	NGC	Δ [95% CI]	L2	NGC	Δ [95% CI]	reported
TwoRoom	91.3 $\pm$3.1	100.0 $\pm$0.0	$+8.7$ $[+4.7, +13.3]$	85.3 $\pm$3.1	100.0 $\pm$0.0	$+14.7$ $[+9.3, +20.7]$	87
Reacher	86.0 $\pm$2.0	84.0 $\pm$3.5	-2.0 $[-9.3, +5.3]$	82.7 $\pm$4.2	81.3 $\pm$11.4	-1.3 $[-9.3, +6.7]$	86
Cube	68.0 $\pm$10.6	78.7 $\pm$7.6	$+10.7$ $[+4.7, +17.3]$	69.3 $\pm$12.1	75.3 $\pm$5.0	$+6.0$ $[-0.7, +12.7]$	74
PushT	91.3 $\pm$2.3	94.0 $\pm$5.3	$+2.7$ $[0.0, +6.0]$	89.3 $\pm$5.0	88.7 $\pm$8.1	-0.7 $[-4.7, +3.3]$	96

C.2 The released LeWM checkpoint

Table 7 repeats the evaluation of Figure 6 using the official checkpoint released by Maes et al. (2026b) in place of ours. The evaluation pairs, planner seeds, success criteria, and step limits remain identical, with each pair seed's graph encoded by the released checkpoint. The pattern of improvements carries over: except at offset 25 on Reacher and PushT, NGC outperforms the L2 cost on every task and offset, with 95% intervals excluding zero. At those two settings, the rates differ by less than 1 point and the intervals include zero. Across nearly all settings, the gains match those of our checkpoint within 4 points; the only exceptions are TwoRoom at offset 25 (where the released

checkpoint starts from a lower L2 success rate and thus gains more) and PushT at offset 100 (where it gains +8.1 versus +15.1).

C.3 TRAINING SEEDS

Table 8 repeats the evaluation of Figure 6 across two additional training seeds of our training recipe. The pairs, planner seeds, success criteria, and step limits are unchanged, and each pair seed's graph is encoded by the corresponding checkpoint. Both additional training seeds closely reproduce the gains of our primary checkpoint: across all tasks and offsets, Δ stays within 4 points of the first seed, except at offset 75 on TwoRoom, where the third seed gains +55.0 compared with +67.2. There, one of the three graphs of the third seed contains two edges that cross the wall (Appendix D), and NGC solves 69% of that pair set, against 93 to 98% of the other eight. Within that affected pair set, NGC solves only 16% of the 33 pairs whose shortest graph path routes through the wall-crossing edges, versus 96% of the remaining 67 pairs.

Table 7: Performance of the released LeWM checkpoint on the pairs of Figure 6 (300 pairs per offset, three planner seeds averaged within each pair). Δ is the mean per-pair difference (NGC minus L2) with its 95% confidence interval; the final column repeats Δ from Table 5 for our checkpoint.

Task	Offset	L2	NGC	Δ [95% CI]	Δ, ours
TwoRoom	25	85.3	99.8	+14.4 [+11.6, +17.6]	+8.3
TwoRoom	50	61.9	99.8	+37.9 [+32.9, +42.9]	+34.1
TwoRoom	75	28.0	95.1	+67.1 [+62.2, +72.0]	+67.2
Reacher	25	97.9	97.4	−0.4 [−1.8, +1.0]	−0.1
Reacher	50	96.2	100.0	+3.8 [+1.9, +5.9]	+4.4
Reacher	100	82.6	100.0	+17.4 [+13.4, +21.6]	+19.4
Cube	25	44.7	55.9	+11.2 [+7.4, +15.1]	+10.3
Cube	50	43.4	57.9	+14.4 [+10.4, +18.6]	+10.4
Cube	100	35.0	51.1	+16.1 [+12.1, +20.2]	+15.8
PushT	25	84.7	83.9	−0.8 [−2.8, +1.1]	+0.7
PushT	50	40.0	57.8	+17.8 [+13.7, +22.1]	+19.8
PushT	100	12.2	20.3	+8.1 [+5.0, +11.3]	+15.1

Table 8: Evaluation across two additional world model training seeds on the pairs of Figure 6 (300 pairs per offset, three planner seeds averaged within each pair). Columns report success rates (%) of the L2 cost and NGC under each seed, along with the mean per-pair difference Δ (NGC minus L2) and its 95% confidence interval. The final column repeats Δ from Table 5 for our primary checkpoint (first seed).

		Second seed			Third seed			First seed
Task	Offset	L2	NGC	Δ [95% CI]	L2	NGC	Δ [95% CI]	Δ
TwoRoom	25	92.2	100.0	+7.8 [+5.4, +10.3]	92.6	100.0	+7.4 [+5.1, +10.0]	+8.3
TwoRoom	50	69.7	99.8	+30.1 [+25.4, +34.9]	66.3	99.2	+32.9 [+27.9, +38.1]	+34.1
TwoRoom	75	31.8	96.1	+64.3 [+59.3, +69.2]	31.4	86.4	+55.0 [+49.7, +60.3]	+67.2
Reacher	25	99.4	99.4	0.0 [−0.7, +0.7]	99.7	99.9	+0.2 [−0.2, +0.7]	−0.1
Reacher	50	95.4	100.0	+4.6 [+2.4, +7.0]	95.6	99.9	+4.3 [+2.2, +6.7]	+4.4
Reacher	100	80.7	100.0	+19.3 [+15.1, +23.7]	81.6	100.0	+18.4 [+14.3, +22.7]	+19.4
Cube	25	48.8	61.3	+12.6 [+8.6, +16.7]	46.3	57.9	+11.6 [+7.9, +15.3]	+10.3
Cube	50	43.7	56.4	+12.8 [+8.9, +16.8]	44.6	53.6	+9.0 [+5.4, +12.7]	+10.4
Cube	100	36.2	53.7	+17.4 [+13.0, +22.0]	34.7	51.3	+16.7 [+12.7, +20.8]	+15.8
PushT	25	90.4	91.3	+0.9 [−1.0, +2.9]	88.8	89.1	+0.3 [−1.8, +2.6]	+0.7
PushT	50	41.6	59.9	+18.3 [+13.7, +23.0]	43.1	61.2	+18.1 [+13.4, +22.9]	+19.8
PushT	100	12.6	24.9	+12.3 [+8.9, +15.9]	13.1	24.4	+11.3 [+7.9, +14.8]	+15.1

D SENSITIVITY TO THE GRAPH CONSTRUCTION

Node sample and neighbor rule. Table 9 varies the default graph construction on the first pair seed of Table 5 along two axes: node sampling and the neighbor connectivity rule. The two resamples each draw $N = 5{,}000$ new random frames from the same pool of eligible episodes while keeping the mutual-15NN rule, whereas the *union* variant keeps the default nodes but connects any pair of nodes where either belongs to the other's 15 nearest neighbors. Across all tasks and offsets, every 95% interval of Δ against the default graph includes zero for both node resamples and the union rule, while at the longer offset of each task every variant retains a higher success rate than the L2 cost. Offline, replacing the mutual rule with the union rule leaves the Spearman correlation between NGC and physical distance within 0.02 of the default (0.94 on TwoRoom, 0.99 on Reacher, 0.73 versus 0.75 on Cube, and 0.54 versus 0.55 on PushT), but introduces longer edges: on TwoRoom, the longest edge spans 47 px of physical path length under the union rule, compared with 29 px under the mutual rule. We use the mutual rule as our default because it enforces stricter locality, aligning with the manifold hypothesis.

Table 9: Sensitivity to node sampling and the neighbor connectivity rule on one pair seed (100 pairs $\times$ 3 planner seeds per cell). All columns except L2 evaluate NGC. The L2 and Default columns (mutual rule, $k = 15$, $5{,}000$ nodes) correspond to the first pair seed of Table 5. Resamples 1 and 2 draw new sets of $5{,}000$ nodes under the mutual rule; the union column uses the default nodes but joins two nodes when either is among the other's 15 nearest neighbors. Each variant cell reports the success rate (%) and, in small type, the mean per-pair difference Δ against the default graph with its 95% confidence interval.

			Node sample (mutual)			Neighbor rule
Task	Offset	L2	Default	resample 1	resample 2	union
TwoRoom	25	91.3	100.0	100.0 0.0	100.0 0.0	100.0 0.0
TwoRoom	75	23.7	95.7	95.3 $-0.3\,[-3.0, +2.3]$	94.7 $-1.0\,[-3.7, +1.3]$	94.3 $-1.3\,[-4.0, +1.0]$
Reacher	25	99.0	99.0	99.3 $+0.3\,[-1.0, +1.7]$	98.7 $-0.3\,[-2.0, +1.3]$	100.0 $+1.0\,[0.0, +2.3]$
Reacher	100	82.7	100.0	100.0 0.0	100.0 0.0	100.0 0.0
Cube	25	43.7	57.0	54.3 $-2.7\,[-8.3, +2.7]$	53.0 $-4.0\,[-9.7, +1.3]$	56.3 $-0.7\,[-5.3, +4.0]$
Cube	100	35.3	49.7	51.0 $+1.3\,[-3.3, +6.0]$	53.0 $+3.3\,[-2.0, +8.7]$	50.3 $+0.7\,[-3.3, +4.7]$
PushT	25	88.7	89.0	90.7 $+1.7\,[-1.3, +5.0]$	88.7 $-0.3\,[-3.3, +2.7]$	88.7 $-0.3\,[-2.0, +1.3]$
PushT	100	13.0	28.3	33.0 $+4.7\,[-2.3, +11.7]$	33.0 $+4.7\,[-2.3, +12.0]$	26.7 $-1.7\,[-6.3, +3.0]$

Graph size and neighborhood size. Table 10 varies the total number of graph nodes N and the neighborhood size k under the mutual rule. On Reacher, Cube, and PushT, NGC is largely insensitive to both hyperparameters: except when N is reduced to $\frac{1}{8}$ of the default (625 nodes), every 95% interval of Δ against the default graph includes zero and lies within -11.3 to $+8.3$ points. On TwoRoom, however, the variants diverge sharply at offset 75: smaller neighborhoods ($k = 5$ and 10) match the high success rate of the default graph, whereas sparser node sets or larger neighborhoods ($k = 30$ and 60) drop by 22 to 51 points.

Figure 10 illustrates a possible geometric explanation for this drop by plotting each TwoRoom graph from Table 10 in physical space, with edges drawn between the rendered (x, y) positions of their endpoint nodes. The five variants whose performance degrades at offset 75 are precisely the five whose graphs contain shortcut edges across the central wall, linking nodes that happen to lie relatively close in the ambient latent space despite being far apart along the feasible path through the doorway; the default graph ($N = 5{,}000, k = 15$) and the smaller-k variants ($k = 5, 10$) contain no such edges. At $\frac{1}{2}N$ ($2{,}500$ nodes), the graph contains a single wall-crossing edge and the success rate is 22 points below the default. This association is consistent with harmful shortcuts, but changing the node set also changes coverage, attachment, and other edges, so the comparison does not

isolate the effect of that edge. Because the mutual-kNN rule connects two nodes only when each falls within the other's k-nearest neighborhood, the spatial reach of an edge scales with the distance to the k-th nearest neighbor. Either reducing the node density N or increasing k expands this reach until it bridges the Euclidean gap across the wall in latent space, causing shortest paths on the graph to cut through the wall.

Table 10: Sensitivity to graph size N and neighborhood size k on one pair seed (100 pairs $\times$ 3 planner seeds per cell). Column headings indicate goal offsets in steps. Each cell reports the success rate (%) and, in small type, the mean per-pair difference Δ against the default graph (5,000 nodes, $k = 15$) with its 95% confidence interval.

	TwoRoom		Reacher	
Variant	25	75	25	100
L2	91.3	23.7	99.0	82.7
NGC (default)	100.0	95.7	99.0	100.0
$N = 625$	100.0 0.0	57.0 $-38.7\,[-48.0, -29.7]$	99.7 $+0.7\,[-0.7, +2.0]$	100.0 0.0
$N = 1{,}250$	100.0 0.0	64.3 $-31.3\,[-40.0, -23.0]$	99.7 $+0.7\,[-0.7, +2.0]$	100.0 0.0
$N = 2{,}500$	100.0 0.0	73.3 $-22.3\,[-30.3, -14.7]$	99.7 $+0.7\,[0.0, +1.7]$	100.0 0.0
$k = 5$	100.0 0.0	96.0 $+0.3\,[-1.0, +2.0]$	99.3 $+0.3\,[-1.0, +1.7]$	100.0 0.0
$k = 10$	99.7 $-0.3\,[-1.0, 0.0]$	98.0 $+2.3\,[+0.3, +5.0]$	99.0 $0.0\,[-1.7, +1.7]$	100.0 0.0
$k = 30$	100.0 0.0	69.3 $-26.3\,[-34.7, -18.3]$	100.0 $+1.0\,[0.0, +2.3]$	100.0 0.0
$k = 60$	95.3 $-4.7\,[-9.0, -1.0]$	45.0 $-50.7\,[-60.0, -41.3]$	99.7 $+0.7\,[-0.7, +2.0]$	100.0 0.0

	Cube		PushT	
Variant	25	100	25	100
L2	43.7	35.3	88.7	13.0
NGC (default)	57.0	49.7	89.0	28.3
$N = 625$	50.3 $-6.7\,[-13.0, -0.7]$	46.3 $-3.3\,[-9.0, +2.0]$	88.7 $-0.3\,[-3.0, +2.0]$	20.3 $-8.0\,[-14.3, -2.0]$
$N = 1{,}250$	56.3 $-0.7\,[-6.0, +5.0]$	47.3 $-2.3\,[-7.0, +2.3]$	86.7 $-2.3\,[-5.7, +0.7]$	22.7 $-5.7\,[-11.3, +0.3]$
$N = 2{,}500$	55.7 $-1.3\,[-6.7, +4.0]$	53.0 $+3.3\,[-1.3, +8.0]$	90.0 $+1.0\,[-1.7, +4.3]$	27.0 $-1.3\,[-8.0, +5.7]$
$k = 5$	56.7 $-0.3\,[-5.3, +4.7]$	51.0 $+1.3\,[-3.0, +5.7]$	89.0 $0.0\,[-4.3, +4.3]$	29.7 $+1.3\,[-5.7, +8.3]$
$k = 10$	58.0 $+1.0\,[-4.0, +6.0]$	49.0 $-0.7\,[-5.3, +4.3]$	90.3 $+1.3\,[-1.3, +4.7]$	29.7 $+1.3\,[-4.7, +7.3]$
$k = 30$	52.7 $-4.3\,[-9.7, +1.0]$	50.3 $+0.7\,[-2.7, +4.3]$	89.0 $0.0\,[-2.0, +2.0]$	26.3 $-2.0\,[-8.7, +4.7]$
$k = 60$	52.7 $-4.3\,[-10.7, +1.7]$	48.7 $-1.0\,[-6.0, +3.7]$	88.7 $-0.3\,[-2.7, +2.0]$	26.3 $-2.0\,[-8.0, +4.0]$

E WHY CAN AN ORACLE PREDICTOR LOWER SUCCESS ON TWOROOM?

In Table 2, replacing the learned predictor with the oracle predictor slightly reduces the success rate of the L2 cost at every offset on TwoRoom. To understand why exact predictions can hurt planning under the L2 cost, we inspect the rollouts and cost maps at offset 25.

Cost geometry near the oracle stopping points. Because the oracle predictor scores each candidate sequence by rendering and encoding its exact terminal state, the cost it assigns matches the ground-truth L2 cost map from Figure 1 (up to the 4 px grid stride). Figure 11 displays the cost maps and rollouts for four pairs where the L2 cost fails with the oracle predictor but succeeds with the learned predictor (left), alongside one-dimensional cost profiles for all 16 oracle failures (right).

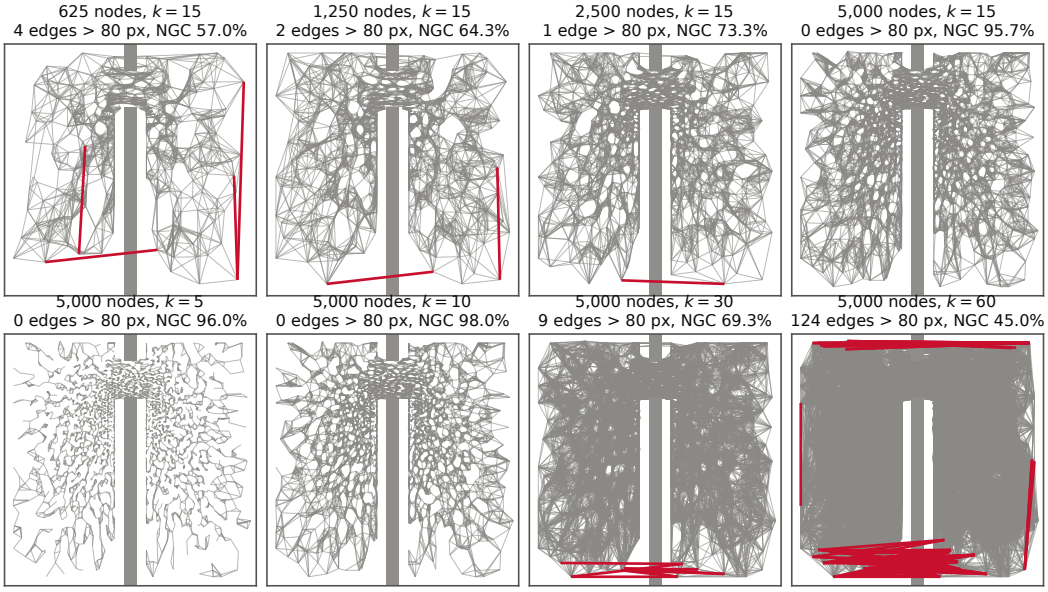


Figure 10: Physical-space visualizations of the TwoRoom graphs from Table 10: nested node subsets and the default graph at $k = 15$ (**top**), and the default $5{,}000$ nodes at $k = 5$, 10, 30, and 60 (**bottom**), with each edge plotted between the physical positions of its endpoint nodes. Red edges highlight shortcuts that connect nodes separated by more than 80 px along the shortest feasible path through the doorway. Titles report the number of such shortcut edges and the resulting NGC success rate at offset 75.

Moving along the straight path from each oracle stopping point toward the goal, the true L2 cost initially rises above its value at the stopping point, dropping below it only within the immediate vicinity of the goal. Consequently, advancing toward the goal from these positions actually worsens the true L2 objective before improving it.

Why exact predictions need not improve success. Of the 16 pairs where the oracle predictor fails under the L2 cost, the learned predictor solves 11. Across all 11 pairs, the oracle rollout stops beside the doorway, 32 to 68 px from the goal (median 50 px), and never approaches closer than 26 px. A possible explanation is that prediction errors make a candidate that advances toward the goal appear cheaper than one that stays near the stopping point, even when their true terminal L2 costs favor staying. Removing such a favorable ranking error can reduce success despite making predictions exact.

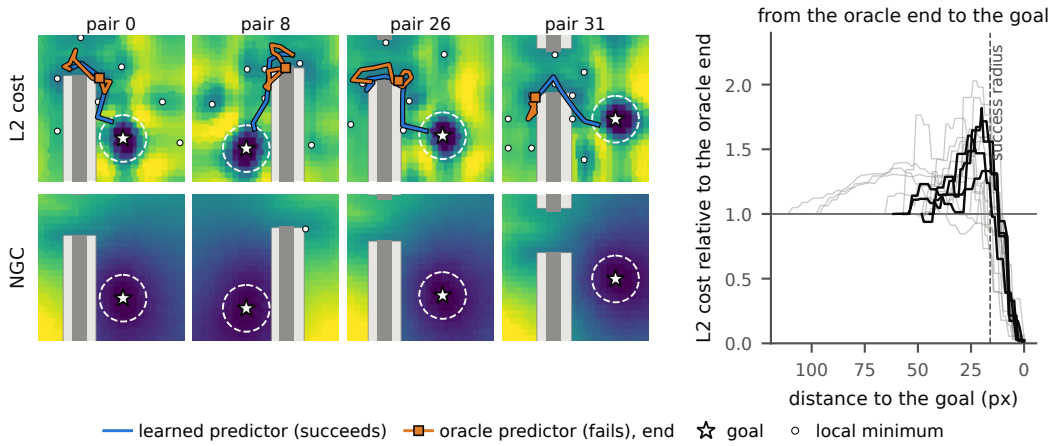


Figure 11: Behavior of the L2 cost with the oracle predictor on TwoRoom at offset 25. **Left:** the first four pairs (in pair-index order) where the L2 cost fails under the oracle predictor but succeeds under the learned predictor, cropped around the stopping point and the goal. Top row: L2 cost maps overlaid with rollouts from both predictors. Bottom row: corresponding NGC cost maps. Each panel is normalized to its own 2nd–98th percentile within the crop; the dashed circle marks the goal success radius, and white dots indicate strict local minima on the grid (points cheaper than all 8 neighbors). **Right:** L2 cost profiles along the straight line from the oracle rollout's stopping point to the goal, normalized by the cost at the stopping point, across all 16 pairs where the L2 cost fails with the oracle predictor (the four pairs shown on the left are highlighted in black).